Course: AI Assisted Coding (23CS002PC304)

Name : P. Samanvith
HT No : 2303A52090
Batch No : 40

Task 1 – Input Validation Check

Task Description: Analyze an AI-generated Python login script for input validation vulnerabilities and suggest secure improvements using regular expressions (re).

Prompt Used:

"Generate a simple Python username and password login program."

Insecure Version

```
# Insecure AI-Generated Code
def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    # Vulnerability: No input validation or sanitization
    if username == "admin" and password == "secret123":
        print("Login successful!")
    else:
        print("Invalid credentials.")

login()
```

Explanation & Vulnerability: The AI generated code accepts any input without validation. It is susceptible to buffer overflow (if inputs are extremely large) and allows special characters which could break downstream systems if this was connected to a database or a shell command.

Secure Version

```
# Secure Refactored Code
import re
```

```
def validate_input(username, password):
    # Username must be alphanumeric, 3-20 characters long
    if not re.match(r"^[a-zA-Z0-9]{3,20}$", username):
        return False
    # Password must be at least 8 chars long and contain alphanumeric characters
    if not re.match(r"^[a-zA-Z0-9@#$%^&+=]{8,50}$", password):
        return False
    return True

def secure_login():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()

    if not validate_input(username, password):
        print("Error: Invalid input format. Please check length and characters.")
        return

    if username == "admin" and password == "secret123":
        print("Login successful!")
    else:
        print("Invalid credentials.")

secure_login()
```

Output:

```
Enter username: admin!@#
Enter password: sec
Error: Invalid input format. Please check length and characters.
```

Task 2 – SQL Injection Prevention

Task Description: Test an AI-generated Python script that performs SQL queries on a database. Identify SQL injection vulnerabilities and refactor using parameterized queries.

Prompt Used:

> "Generate a Python script using SQLite to fetch user details based on an input username."

Insecure Version

```
# Insecure AI-Generated Code
import sqlite3

def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Vulnerability: String concatenation leads to SQL Injection
    query = f"SELECT * FROM users WHERE username = '{username}'"
    cursor.execute(query)

    result = cursor.fetchall()
    conn.close()
    return result

# A malicious input like: admin' OR '1'='1
print(get_user("admin' OR '1'='1"))
```

Explanation & Vulnerability:

The AI used an f-string to insert the variable directly into the SQL query. If a user inputs ' OR '1'='1, the query evaluates to SELECT * FROM users WHERE username = '' OR '1'='1', which is always true, bypassing authentication and returning all user records.

Secure Version

```
# Secure Refactored Code
import sqlite3

def secure_get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Secure: Using parameterized queries (Prepared Statements)
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    result = cursor.fetchall()
    conn.close()
```

```
    return result

# The malicious input is treated purely as a string literal, not executable code
print(secure_get_user("admin' OR '1'='1"))
```

Output:

```
[]  # Returns empty array instead of exposing the whole database
```

Task 3 – Cross-Site Scripting (XSS) Check

Task Description: Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities and implement secure measures.

Prompt Used:

> "Create a simple HTML feedback form with JavaScript that displays the submitted feedback on the page below the form."

Insecure Version

```
<!-- Insecure AI-Generated Code -->
<!DOCTYPE html>
<html>
<body>
    <input type="text" id="feedbackInput" placeholder="Enter feedback">
    <button onclick="submitFeedback()">Submit</button>
    <div id="output"></div>

    <script>
        function submitFeedback() {
            let feedback = document.getElementById("feedbackInput").value;
            // Vulnerability: Directly rendering untrusted input using innerHTML
            document.getElementById("output").innerHTML = "You submitted: " + feedback;
        }
    </script>
</body>
</html>
```

Explanation & Vulnerability:

The script uses innerHTML to display the user's input. If an attacker submits <img src=x onerror=alert('XSS')>, the browser will execute the script. This allows session hijacking or malicious redirects.

Secure Version

```
<!-- Secure Refactored Code -->
<!DOCTYPE html>
<html>
<head>
  <!-- Content Security Policy to restrict inline scripts -->
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' 'unsafe-inline';">
</head>
<body>
  <input type="text" id="feedbackInput" placeholder="Enter feedback">
  <button onclick="submitFeedback()">Submit</button>
  <div id="output"></div>

  <script>
    function submitFeedback() {
      let feedback = document.getElementById("feedbackInput").value;
      // Secure: Using textContent which safely escapes HTML entities
      document.getElementById("output").textContent = "You submitted: " + feedback;
    }
  </script>
</body>
</html>
```

Output:

(When submitting `<script>alert(1)</script>`)
Displayed on screen: You submitted: <script>alert(1)</script>
(The script is NOT executed, just displayed as plain text)

Task 4 – Real-Time Application: Security Audit of AI-Generated Code

Task Description: Perform a security audit to detect vulnerabilities in an AI-generated File

Upload snippet. Suggest secure coding practices and compare insecure vs secure versions.

Prompt Used:

> "Write a Python Flask endpoint for users to upload profile pictures to a specific folder."

Insecure vs Secure Comparison

Insecure Version:

```
from flask import Flask, request
import os

app = Flask(__name__)
UPLOAD_FOLDER = 'uploads/'

@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']
    # Vulnerability 1: Trusts the filename provided by the user (Path Traversal)
    # Vulnerability 2: Accepts any file extension (e.g., .exe, .sh, .py)
    file.save(os.path.join(UPLOAD_FOLDER, file.filename))
    return "File uploaded successfully"
```

Security Audit Findings:

1. **Directory Traversal:** If file.filename is ../../../etc/passwd, the server might overwrite critical system files.
2. **Unrestricted File Upload:** Attackers can upload malicious scripts (e.g., PHP, Python, executable binaries) and run them on the server.

Secure Version:

```
from flask import Flask, request, abort
from werkzeug.utils import secure_filename
import os

app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = 'uploads/'
# Secure Practice 1: Restrict allowed extensions
app.config['ALLOWED_EXTENSIONS'] = {'png', 'jpg', 'jpeg', 'gif'}
# Secure Practice 2: Limit file size to 2MB
```

```
app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024

def allowed_file(filename):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in app.config['ALLOWED_EXTENSIONS']

@app.route('/upload', methods=['POST'])
def secure_upload_file():
    if 'file' not in request.files:
        return abort(400, "No file part")

    file = request.files['file']
    if file.filename == '':
        return abort(400, "No selected file")

    if file and allowed_file(file.filename):
        # Secure Practice 3: Strip path traversal characters
        filename = secure_filename(file.filename)
        file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
        return "Secure file upload successful"
    else:
        return abort(400, "Invalid file type")
```

Output:

```
Attempting to upload 'malicious_script.py'...
Response: 400 Bad Request - Invalid file type

Attempting to upload '../../../image.png'...
Response: Secure file upload successful (Saved as 'image.png' in correct folder)
```

Task 5 – Insecure Data Serialization Check

Task Description: Evaluate AI-generated code that uses pickle for saving user data, explain why untrusted pickle data is dangerous, and replace it with safer formats like JSON.

Prompt Used:

> "Write a Python script that saves and loads a user dictionary using pickle."

Insecure Version

```
# Insecure AI-Generated Code
import pickle

def save_user(user_data, filename):
    with open(filename, 'wb') as f:
        pickle.dump(user_data, f)

def load_user(filename):
    with open(filename, 'rb') as f:
        # Vulnerability: Loading untrusted data using pickle
        return pickle.load(f)

data = {"username": "john_doe", "role": "user"}
save_user(data, "user.pkl")
loaded_data = load_user("user.pkl")
```

Explanation & Danger of pickle:

The pickle module is fundamentally insecure for untrusted data. When unpickling, Python can reconstruct arbitrary objects. An attacker can craft a malicious pickle payload using the __reduce__ method to execute arbitrary system commands (RCE - Remote Code Execution) immediately upon pickle.load() being called.

Secure Version

```
# Secure Refactored Code
import json

def secure_save_user(user_data, filename):
    with open(filename, 'w') as f:
        # Secure: JSON serializes data, not executable objects
        json.dump(user_data, f)

def secure_load_user(filename):
    with open(filename, 'r') as f:
        # Secure: JSON only parses native, harmless data structures
        return json.load(f)

data = {"username": "john_doe", "role": "user"}
```

```
secure_save_user(data, "user.json")
loaded_data = secure_load_user("user.json")
print("Data loaded securely:", loaded_data)
```

Output:

```
Data loaded securely: {'username': 'john_doe', 'role': 'user'}
```